

Evidencia

- Captura de psql --version y de \l mostrando la base catalogo

```
(venv) ➔ catologo-musical-django git:(feature/postgres) X psql --version
psql (PostgreSQL) 18.6
(venv) ➔ catologo-musical-django git:(feature/postgres) X sudo -u postgres psql
[sudo] contraseña para jesus:
psql (18.6)
Digite «help» para obtener ayuda.

postgres=# \l
                                Listado de base de datos
 e | Nombre | Dueño | Codificación | Proveedor de locale | Collate | Ctyp
---+-----+-----+-----+-----+-----+-----
 catalogo | catalogo_user | UTF8 | libc | es_MX.UTF-8 | es_MX.U
 TF-8 | | | | | | 
 catalogo_respaldo | catalogo_user | UTF8 | libc | es_MX.UTF-8 | es_MX.U
 TF-8 | | | | | | 
 postgres | postgres | UTF8 | libc | es_MX.UTF-8 | es_MX.U
 TF-8 | | | | | | 
 template0 | postgres | UTF8 | libc | es_MX.UTF-8 | es_MX.U
 TF-8 | | | =c/postgres | | | 
 | | | | | | 
 | | | postgres=CtC/postgres | | | 
 template1 | postgres | UTF8 | libc | es_MX.UTF-8 | es_MX.U
 TF-8 | | | =c/postgres | | | 
 | | | | | | 
 | | | postgres=CtC/postgres | | | 
(5 filas)
postgres=#
```

- Captura de los conteos en el shell y de la API de DRF respondiendo con Postgres.

```
(venv) ➔ catologo-musical-django git:(feature/postgres) X python manage.py shell
14 objects imported automatically (use -v 2 for details).

Python 3.14.7 (main, Aug 10 2026, 00:00:00) [GCC 16.1.1 20260515 (Red Hat 16.1.1-2)] on linux
Type "help", "copyright", "credits" or "license" for more information.
(InteractiveConsole)
>>> from catologo.models import Cancion, Playlist
>>> Cancion.objects.count()
28359
>>> Playlist.objects.count()
472
>>> Cancion.playlists.through.objects.count()
32253
>>>
```

```
(venv) ➔ catologo-musical-django git:(feature/postgres) X python manage.py shell
14 objects imported automatically (use -v 2 for details).

Python 3.14.7 (main, Aug 10 2026, 00:00:00) [GCC 16.1.1 20260515 (Red Hat 16.1.1-2)] on linux
Type "help", "copyright", "credits" or "license" for more information.
(InteractiveConsole)
>>> from catologo.models import Cancion, Playlist
>>> Cancion.objects.count()
28359
>>> Playlist.objects.count()
472
>>> Cancion.playlists.through.objects.count()
32253
>>>
```

Cancion List

OPTIONS GET

1 2 3 ... 1418

GET /api/canciones/

HTTP 200 OK
Allow: GET, POST, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept

```
{
  "count": 28359,
  "next": "http://127.0.0.1:8000/api/canciones/?page=2",
  "previous": null,
  "results": []
}
```

- El error que apareció al cargar el CSV y cómo lo resolvieron.

Al probar la conexión con `python manage.py dbshell`, apareció el error

```
(venv) ➔ catologo-musical-django git:(feature/postgres) ✕
python manage.py dbshell
psql: error: falló la conexión al servidor en «localhost» (
::1), puerto 5432: FATAL: la autenticación Ident falló p
ara el usuario «catologo_user»
CommandError: "psql -U catologo_user -h localhost -p 5432 c
atologo" returned non-zero exit status 2.
```

Investigando el motivo, encontramos que PostgreSQL en Linux usa por defecto el método de autenticación peer o ident para las conexiones locales, el cual exige que el usuario del sistema operativo coincida con el usuario de PostgreSQL. Como en la terminal estábamos como `jesus` pero Django intentaba conectarse como `catologo_user`, PostgreSQL rechazaba la conexión.

Para resolverlo, editamos el archivo de configuración de PostgreSQL

- `sudo nano /etc/postgresql/18/main/pg_hba.conf`

Ahí cambiamos el método de autenticación de las conexiones locales de `ident` a `scram-sha-256`, para que PostgreSQL acepte la autenticación por contraseña en lugar de exigir que coincidan los usuarios del sistema. Después reiniciamos el servicio para que los cambios surtieran efecto y volvimos a probar la conexión

```
(venv) ➔ catologo-musical-django git:(feature/postgres) ✕
python manage.py dbshell
psql (18.6)
Digite «help» para obtener ayuda.

catologo=> ^C
catologo=> exit()
catologo-> ^C
catologo=> \q
(venv) ➔ catologo-musical-django git:(feature/postgres) ✕
```

- Las respuestas de los retos que alcanzaron a resolver.

Reto 1

Anoten los cuatro números (dos consultas por dos motores).

1. Sqlite:

1,111

1,111

2. Postgres

29

1,111

¿Por qué en SQLite contains e iconains dan casi lo mismo y en Postgres no?

Debido a que sqlite no distingue entre mayúsculas y minúsculas, postgres si, por lo que al pasar el argumento diferente con contains e iconain, para sqlite la consulta es la misma mientras que para postgres es diferente.

Revisen la vista de búsqueda que hicieron el jueves pasado: ¿usaron contains o iconains? ¿Su búsqueda se comporta igual en Postgres?

Para la búsqueda en general con q si se uso iconains en titulo y artista, y si se comporta igual por eso cuando se busca love en minúscula igualmente se obtendrán los 1,111 resultados

Reto 2

```
>>> Cancion.objects.filter(playlists__genero="pop").count()
```

5507

```
>>> Cancion.objects.filter(playlists__genero="pop").distinct().count()
```

5132

Expliquen con sus palabras la diferencia entre ambos números.

El primer numero trae todas las playlist que tienen genero de pop, aunque sea repetida, la segunda descarta las repetidas

Ahora prueben algo que solo existe en Postgres:

- Postgres

```
>>> Playlist.objects.order_by("genero").distinct("genero").values_list("genero", flat=True)
```

```
<QuerySet ['edm', 'latin', 'pop', 'rap', 'r&b', 'rock', 'Rock']>
```

Cambien a SQLite y ejecuten la misma línea. ¿Qué error aparece?

- Sqlite

django.db.utils.NotSupportedError: DISTINCT ON fields is not supported by this database backend

Esto sucede porque según lo que investigue postgres permite hacer un distinct en campos (columnas) mientras que sqlite no, también pasa con MySQL

Si su API de DRF usara distinct("genero"), ¿qué pasaría el día que alguien la corriera con SQLite? ¿Qué conclusión sacan sobre usar funciones exclusivas de un motor?

Rompería por el mismo error que sqlite no permite función distinct a campos, este caso por buscar el campo genero, se debería investigar antes de escoger un motor, para evitar problemas

Reto 3

Sin db_index=True

-> Seq Scan on catalogo_cancion (cost=0.00..859.49 rows=34 width=101) (actual time=0.033..5.684 rows=34.00 loops=1)

Filter: ((artista)::text = 'Ed Sheeran'::text)

Rows Removed by Filter: 28325

Buffers: shared hit=505

Planning:

Buffers: shared hit=187

Planning Time: 0.903 ms

Execution Time: 5.944 ms

Con db_index=True

-> Bitmap Heap Scan on catalogo_cancion (cost=4.55..111.67 rows=34 width=101) (actual time=0.078..0.294 rows=34.00 loops=1)

Recheck Cond: ((artista)::text = 'Ed Sheeran'::text)

Heap Blocks: exact=27

Buffers: shared hit=27 read=2

-> Bitmap Index Scan on catalogo_cancion_artista_fa8dd6e9_like (cost=0.00..4.54 rows=34 width=0) (actual time=0.051..0.052 rows=34.00 loops=1)

Index Cond: ((artista)::text = 'Ed Sheeran'::text)

Index Searches: 1

Buffers: shared read=2

Planning:

Buffers: shared hit=215 read=2

Planning Time: 0.928 ms

Execution Time: 0.597 ms

Repitan el explain. ¿Cambió el tipo de búsqueda? ¿Cambió el tiempo?

Si, cambio de seq scan a bitmap, ahora postgres usa el el indice que agregamos y ya no recorre todas las canciones is no va directo a las 34 que cumplen la condición

Ahora prueben con artista_icontains="sheeran". ¿Postgres usa el índice? ¿Por qué creen que no?

Sort (cost=930.41..930.42 rows=3 width=101) (actual time=18.726..18.781 rows=34.00 loops=1)

Sort Key: popularidad DESC

Sort Method: quicksort Memory: 30kB

Buffers: shared hit=505

-> Seq Scan on catalogo_cancion (cost=0.00..930.38 rows=3 width=101) (actual time=0.013..18.643 rows=34.00 loops=1)

Filter: (upper((artista)::text) ~~ '%SHEERAN% '::text)

Rows Removed by Filter: 28325

Buffers: shared hit=505

Planning Time: 0.138 ms

Execution Time: 18.867 ms

Ahora con icontains postgres no usa el indice creado, vuelve a hacer seq scan.

Reto 4

(venv) → catalogo-musical-django git:(feature/postgres) ✗ python manage.py shell

14 objects imported automatically (use -v 2 for details).

Python 3.14.7 (main, Aug 10 2026, 00:00:00) [GCC 16.1.1 20260515 (Red Hat 16.1.1-2)] on linux

Type "help", "copyright", "credits" or "license" for more information.

(InteractiveConsole)

>>> from catalogo.models import Cancion, Playlist

```
>>> Cancion.objects.count()
```

28359

```
>>> Playlist.objects.count()
```

472

```
>>> Cancion.playlists.through.objects.count()
```

32253

Verifiquen los conteos en el shell. ¿Coinciden con los de catalogo?

Si coinciden

¿Qué fue más rápido, cargar_csv o loaddata? ¿Por qué creen?

Loaddata lo senti un poco mas rápido, yo creo que es porque aquí el loaddata lo toma del json que ya estaba "limpio" y filtrado y en cargar_csv tuvimos que hacer un proceso de limpiar los datos

- Respuesta breve: ¿cuántas líneas de vistas, serializers o URLs cambiaron y qué les dice eso sobre el ORM?

No se cambio ni una linea de código, solo se cambio lo de la base de datos para poder alternar entre postgres y sqlite y las variables de entorno, esto nos dice que la abstracción del ORM de django es buena porque con el mismo código funciona igual en postgres como en sqlite sin embargo cambia dependiendo el gestor de base de datos como lo vimos con las mayúsculas y minúsculas y con el distinct en campos

- Declaración de uso de IA: qué le pidieron, qué modificaron o revisaron del resultado y por qué, o declaración explícita de que no la usaron.

Se utilizó IA como apoyo para investigar y comprender algunos problemas técnicos durante la migración de SQLite a PostgreSQL. En particular, se pidió ayuda para entender el error de autenticación de PostgreSQL y para identificar qué configuración debía modificarse en pg_hba.conf. Después se revisó el resultado y se modificó manualmente el archivo mediante nano, cambiando la autenticación de ident a scram-sha-256 para las conexiones locales. También se verificó que PostgreSQL reiniciara correctamente y que Django pudiera conectarse mediante dbshell.